

October 17

Problem 1: House Robber

Problem Statement:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, but adjacent houses have security systems connected, so if two adjacent houses are broken into on the same night, the system will automatically alert the police.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Link to problem:

<https://leetcode.com/problems/house-robber/description/>

Example 1:

- Input: `nums = [1, 2, 3, 1]`
- Output: 4
- Explanation: Rob house 1 (1) and house 3 (3), for a maximum of $1 + 3 = 4$.

Example 2:

- Input: `nums = [2, 7, 9, 3, 1]`
 - Output: 12
 - Explanation: Rob house 1 (2), house 3 (9), and house 5 (1), for a maximum of $2 + 9 + 1 = 12$.
-

Solution:

```
import java.util.*;
class Solution {
    public int rob(int[] nums) {
        if (nums.length == 0) return 0;
        if (nums.length == 1) return nums[0];

        int prev1 = 0, prev2 = 0;

        for (int num : nums) {
            int temp = prev1;
            prev1 = Math.max(prev2 + num, prev1);
            prev2 = temp;
        }
    }
}
```

```
        return prev1;
    }
}
```

Explanation:

- **Initial Setup:**

- **Edge case checks:**

- If there are no houses (`nums.length == 0`), return 0 because there's nothing to rob.
 - If there's only one house (`nums.length == 1`), return the amount in that house since it's the only option.

- **Dynamic Programming Variables:**

- `prev1`: Stores the maximum amount robbed up to the **previous house**.
 - `prev2`: Stores the maximum amount robbed up to the **house before the previous one**.

- **Iterating through the houses:**

- For each house `i`, you have two choices:
 - **Skip the house:** This means you keep the maximum money robbed so far, which is `prev1`.
 - **Rob the house:** This means you add the value of the current house (`num`) to the maximum robbed up to the house two places earlier (`prev2 + num`).
 - The new value of `prev1` is set to the maximum of these two choices.

- **Updating the variables:**

- Once the decision for house `i` is made, update `prev2` to hold the old value of `prev1` (which is now outdated) for the next iteration.

- **Final Answer:**

- After processing all houses, `prev1` holds the maximum amount of money that can be robbed without triggering the alarm.
-

Time Complexity:

- **$O(n)$** : We iterate over all `n` houses once.

Space Complexity:

- **$O(1)$** : We use only two extra variables (`prev1` and `prev2`) regardless of the input size.
-

Problem 2: SubSets

Problem Statement:

Given an integer array `nums` of unique elements, return all possible subsets (the power set).

The solution set must not contain duplicate subsets. Return the solution in any order.

Link to problem:

<https://leetcode.com/problems/subsets/description/>

Example 1:

- **Input:** `nums = [1, 2, 3]`
- **Output:** `[[], [1], [2], [1, 2], [3], [1, 3], [2, 3], [1, 2, 3]]`

Example 2:

- **Input:** `nums = [0]`
 - **Output:** `[[], [0]]`
-

Solution:

```
import java.util.*;

class Solution {
    public List<List<Integer>> subsets(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        generateSubsets(0, nums, new ArrayList<>(), result);
        return result;
    }
    private void generateSubsets(int index, int[] nums, List<Integer> current,
        List<List<Integer>> result) {
        // Base Case: When index reaches the end of the array, add the current subset to result
        if (index == nums.length) {
            result.add(new ArrayList<>(current)); // Add the current subset
            return;
        }
        // Recursive Case 1: Exclude the current element at index
        generateSubsets(index + 1, nums, current, result);
        // Recursive Case 2: Include the current element at index
        current.add(nums[index]);
        generateSubsets(index + 1, nums, current, result);
        // Backtrack to remove the last element added to the current subset
        current.remove(current.size() - 1);
    }
}
```

Explanation:

1. **Base Case:**
 - The recursion stops when the index reaches the length of the array (`index == nums.length`).
 - At this point, we add the current subset to the result list.
 2. **Recursive Case 1 (Exclude the current element):**
 - We make the first recursive call without adding the current element (`nums[index]`) to the current subset.
 - This explores the branch where the current element is excluded.
 3. **Recursive Case 2 (Include the current element):**
 - After excluding the current element, we backtrack and include the element (`nums[index]`) in the subset, then make the second recursive call.
 - This explores the branch where the current element is included.
 4. **Backtracking:**
 - After including the element in the subset, we must remove it (backtrack) to ensure that the subsequent recursive calls work with the correct subset.
-

Example :

Consider `nums = [1, 2, 3]`. The recursion works as follows:

1. Start at index 0 with an empty subset `[]`.
 - Exclude 1:
 - Exclude 2:
 - Exclude 3: Subset is `[]`.
 - Include 3: Subset is `[3]`.
 - Include 2:
 - Exclude 3: Subset is `[2]`.
 - Include 3: Subset is `[2, 3]`.
 - Include 1:
 - Exclude 2:
 - Exclude 3: Subset is `[1]`.
 - Include 3: Subset is `[1, 3]`.
 - Include 2:
 - Exclude 3: Subset is `[1, 2]`.
 - Include 3: Subset is `[1, 2, 3]`.

The final result is: `[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]`.

Time Complexity: $O(n * 2^n)$: We still generate 2^n subsets (since each element can either be included or excluded), and copying each subset takes $O(n)$ time.

Space Complexity: $O(n)$: The recursion depth is at most n , and the space used for storing the current subset is also proportional to n .
